

EXPERIMENT-6

SACHIN SINGH

24UADS1045

Objective :-

WAP to train and evaluate a Recurrent Neural Network using PyTorch Library to predict the next value in a sample time series dataset.

Description Of the Model :-

1.Type of Model:-

The model is a Long Short-Term Memory (LSTM) neural network.

LSTM is a type of Recurrent Neural Network (RNN) specifically designed to handle sequential and time-dependent data.

LSTM networks are capable of learning:

- Temporal dependencies in data
- Long-term patterns in sequences
- Trends and cyclic behavior in time series

2.Dataset Description:-

The dataset used is a sine wave generated using NumPy.

It represents a continuous and smooth time series pattern.

Before training:

- The data is normalized to improve model performance
- A sliding window approach is applied to create input-output pairs

Each input consists of a fixed-length sequence (e.g., 20 time steps), and the target is the next value in the sequence.

3.Data Preprocessing:-

The preprocessing steps include:

- Normalization of data (mean = 0, standard deviation = 1)
- Conversion of data into sequences using a sliding window
- Splitting the dataset into training and testing sets
- Reshaping data into (samples, sequence length, features) format required for LSTM

4.Model Architecture:-

The model consists of the following layers:

- LSTM Layer
 - Input size = 1 (univariate data)
 - Hidden size = 64
 - Learns temporal dependencies using memory cells and gates

- Fully Connected (Linear) Layer
 - Takes output from LSTM
 - Produces final prediction (next value)

Only the output from the last time step of the LSTM is used for prediction.

5.Training Process:-

The model is trained using:

- Loss Function
 - Mean Squared Error (MSE)
 - Measures difference between predicted and actual values
- Optimizer
 - Adam optimizer
 - Adjusts weights efficiently during training
- Epochs
 - The model is trained for multiple epochs
 - In each epoch:
 - Forward pass is performed
 - Loss is calculated
 - Backpropagation updates weights

6.Model Evaluation:-

After training, the model is evaluated on test data.

Predictions are compared with actual values using the following metrics:

- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Error (MAE)
- R-squared Score (R^2)

These metrics indicate how well the model performs on unseen data.

7.Output and Visualization:-

The model generates predictions for the test dataset.

Performance is visualized using graphs such as:

- Loss vs Epochs
- Actual vs Predicted values
- Prediction error plot
- Residual analysis

These graphs help in understanding model accuracy and behavior.

Source Code :-

```
import torch

import torch.nn as nn

import numpy as np

import matplotlib.pyplot as plt

# 1. Create Time Series Data

t = np.linspace(0, 100, 500)

data = np.sin(t)


# Normalize

data = (data - np.mean(data)) / np.std(data)

# 2. Prepare Dataset

seq_length = 20


X = []

y = []


for i in range(len(data) - seq_length):

    X.append(data[i:i+seq_length])

    y.append(data[i+seq_length])


X = np.array(X)

y = np.array(y)

# Convert to tensors

X = torch.FloatTensor(X).unsqueeze(-1) # (samples, seq_len, 1)

y = torch.FloatTensor(y)


# 3. Train/Test Split

train_size = int(0.8 * len(X))
```

```

X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

# 4. LSTM Model

# =====

class LSTMModel(nn.Module):

    def __init__(self):
        super(LSTMModel, self).__init__()

        self.lstm = nn.LSTM(input_size=1, hidden_size=64, batch_first=True)

        self.fc = nn.Linear(64, 1)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = out[:, -1, :] # last time step
        out = self.fc(out)
        return out.squeeze()

model = LSTMModel()

# 5. Loss & Optimizer

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.005)

# 6. Training

# =====

epochs = 30

losses = []
all_preds = []

for epoch in range(epochs):
    model.train()

```

```
optimizer.zero_grad()
```

```
output = model(X_train)
```

```
loss = criterion(output, y_train)
```

```
loss.backward()
```

```
optimizer.step()
```

```
losses.append(loss.item())
```

```
all_preds.append(output.detach().numpy())
```

```
# Detailed logging
```

```
print(f"\nEpoch {epoch+1}/{epochs}")
```

```
print("Input Sample:", X_train[0].flatten().numpy())
```

```
print("Target:", y_train[0].item())
```

```
print("Predicted:", output[0].item())
```

```
print("Loss:", loss.item())
```

```
# 7. Evaluation
```

```
# =====
```

```
model.eval()
```

```
with torch.no_grad():
```

```
    test_preds = model(X_test)
```

```
    test_loss = criterion(test_preds, y_test)
```

```
print("\nTest Loss:", test_loss.item())
```

```
from sklearn.metrics import r2_score
```

```
print("R2 Score:", r2_score(y_test.numpy(), test_preds.numpy()))
```

```
from sklearn.metrics import mean_squared_error
```

```
import numpy as np
```

```
rmse = np.sqrt(mean_squared_error(y_test.numpy(), test_preds.numpy()))
```

```
print("RMSE:", rmse)
```

```
accuracy = (1 - rmse) * 100
```

```
print("Approx Accuracy (%):", accuracy)
```

```
# 8. Graphs
```

```
# =====
```

```
# 1. Loss Curve
```

```
plt.figure()
```

```
plt.plot(losses)
```

```
plt.title("Loss vs Epochs")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.show()
```

```
# 2. Actual vs Predicted
```

```
plt.figure()
```

```
plt.plot(y_test.numpy(), label="Actual")
```

```
plt.plot(test_preds.numpy(), label="Predicted")
```

```
plt.legend()
```

```
plt.title("Actual vs Predicted")
```

```
plt.show()
```

```
# 3. Actual vs Predicted Scatter Plot
```

```
plt.figure()
```

```
plt.scatter(y_true, y_pred)
```

```
plt.xlabel("Actual Values")
```

```
plt.ylabel("Predicted Values")
```

```
plt.title("Actual vs Predicted Scatter")
```

```
plt.show()
```

```
# 4. Error Plot
```

```
error = y_test.numpy() - test_preds.numpy()
```

```
plt.figure()
plt.plot(error)
plt.title("Prediction Error")
plt.show()
```

5. Residual Distribution

```
y_true = y_test.numpy()
y_pred = test_preds.numpy()
residuals = y_true - y_pred
plt.figure()
plt.hist(residuals, bins=30)
plt.title("Residual Distribution")
plt.xlabel("Error")
plt.ylabel("Frequency")
plt.show()
```

6. Residual vs Predictions

```
plt.figure()
plt.scatter(y_pred, residuals)
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.title("Residuals vs Predictions")
plt.show()
```

Output:-

Epoch 1/30

Input Sample: [-1.2188824e-03 2.7990270e-01 5.4977202e-01 7.9758728e-01

1.0134293e+00 1.1886587e+00 1.3162618e+00 1.3911312e+00

1.4102699e+00 1.3729119e+00 1.2805527e+00 1.1368890e+00

9.4767112e-01 7.2047269e-01 4.6438766e-01 1.8966614e-01

-9.2695810e-02 -3.7139627e-01 -6.3527989e-01 -8.7378448e-01]

Target: -1.0773634910583496

Predicted: -0.013274814002215862

Loss: 0.991046130657196

Epoch 2/30

Input Sample: [-1.2188824e-03 2.7990270e-01 5.4977202e-01 7.9758728e-01

1.0134293e+00 1.1886587e+00 1.3162618e+00 1.3911312e+00

1.4102699e+00 1.3729119e+00 1.2805527e+00 1.1368890e+00

9.4767112e-01 7.2047269e-01 4.6438766e-01 1.8966614e-01

-9.2695810e-02 -3.7139627e-01 -6.3527989e-01 -8.7378448e-01]

Target: -1.0773634910583496

Predicted: -0.046669427305459976

Loss: 0.9199407696723938

Epoch 3/30

Input Sample: [-1.2188824e-03 2.7990270e-01 5.4977202e-01 7.9758728e-01

1.0134293e+00 1.1886587e+00 1.3162618e+00 1.3911312e+00

1.4102699e+00 1.3729119e+00 1.2805527e+00 1.1368890e+00

9.4767112e-01 7.2047269e-01 4.6438766e-01 1.8966614e-01

-9.2695810e-02 -3.7139627e-01 -6.3527989e-01 -8.7378448e-01]

Target: -1.0773634910583496

Predicted: -0.07406936585903168

Loss: 0.8468055129051208

.

.
.
.
.

Epoch 29/30

Input Sample: [-1.2188824e-03 2.7990270e-01 5.4977202e-01 7.9758728e-01

1.0134293e+00 1.1886587e+00 1.3162618e+00 1.3911312e+00

1.4102699e+00 1.3729119e+00 1.2805527e+00 1.1368890e+00

9.4767112e-01 7.2047269e-01 4.6438766e-01 1.8966614e-01

-9.2695810e-02 -3.7139627e-01 -6.3527989e-01 -8.7378448e-01]

Target: -1.0773634910583496

Predicted: -1.2656326293945312

Loss: 0.010954814963042736

Epoch 30/30

Input Sample: [-1.2188824e-03 2.7990270e-01 5.4977202e-01 7.9758728e-01

1.0134293e+00 1.1886587e+00 1.3162618e+00 1.3911312e+00

1.4102699e+00 1.3729119e+00 1.2805527e+00 1.1368890e+00

9.4767112e-01 7.2047269e-01 4.6438766e-01 1.8966614e-01

-9.2695810e-02 -3.7139627e-01 -6.3527989e-01 -8.7378448e-01]

Target: -1.0773634910583496

Predicted: -1.2627513408660889

Loss: 0.009806524962186813

Description Of Code :-

1.Data Generation:-

The code begins by generating a time series dataset using a sine function.

A sequence of values is created using NumPy over a defined range.

This synthetic dataset is used to simulate real-world time-dependent data.

2.Data Normalization:-

The generated data is normalized using mean and standard deviation.

This step ensures that all values are on a similar scale, which helps in faster and more stable model training.

3.Sequence Preparation:-

A sliding window technique is applied to convert the time series into input-output pairs. Each input consists of a fixed number of consecutive values (sequence length), and the target is the next value in the sequence. This prepares the data in a supervised learning format.

4.Conversion to Tensors:-

The input and target data are converted into PyTorch tensors. The input is reshaped into three dimensions (samples, sequence length, features) as required by the LSTM model.

5.Train-Test Split:-

The dataset is divided into training and testing sets. A portion of the data (typically 80 percent) is used for training, while the remaining data is used for evaluation.

6.Model Definition:-

A class is defined for the LSTM model using PyTorch.

The model consists of:

- An LSTM layer to process sequential data
- A fully connected layer to produce the final output

7.Loss Function and Optimizer:-

The Mean Squared Error (MSE) loss function is defined to measure prediction error. The Adam optimizer is initialized to update model weights during training.

8.Training Loop:-

A loop is used to train the model for a fixed number of epochs.

In each epoch:

- The model is set to training mode
- Gradients are reset to zero
- Forward pass generates predictions
- Loss is computed between predicted and actual values
- Backward pass calculates gradients
- Optimizer updates model weights

9.Evaluation Phase:-

After training, the model is switched to evaluation mode. Predictions are made on the test dataset without updating weights. The test loss is calculated to assess model performance.

10.Performance Metrics Calculation:-

Additional evaluation metrics such as R-squared score, Mean Absolute Error, and Root Mean Squared Error are computed. These metrics provide a better understanding of prediction accuracy.

11.Visualization of Results:-

Multiple graphs are generated using Matplotlib to analyze model performance:

- Loss vs epochs to observe training progress
- Actual vs predicted values to compare outputs

- Error plot to visualize prediction differences
- Residual plots to examine model behavior

12.Final Output:-

The code outputs:

- Trained model predictions
- Loss values during training
- Evaluation metrics
- Graphical visualizations

Summary of Performance Evaluation :-

The model was trained for 30 epochs, and by the final epoch it demonstrated strong learning of the time series pattern. The predicted value (-1.2627) is close to the actual target value (-1.0773), indicating that the model has effectively captured the underlying sequence behavior. The training loss at the final epoch is 0.0098, which is relatively low and shows that the model has minimized prediction error during training.

On the test dataset, the model achieved a loss of 0.0082, which is slightly lower than the training loss. This suggests that the model generalizes well to unseen data and does not suffer from overfitting.

The R2 score of 0.9917 indicates an excellent fit, meaning that over 99 percent of the variance in the data is explained by the model. This reflects very high prediction accuracy for a regression task.

The RMSE value of 0.0906 is low, indicating that the average prediction error is small and the model's outputs are very close to actual values.

An approximate accuracy of 90.94 percent further supports that the model performs well, although accuracy is not a standard metric for regression problems.

Overall, the model shows high accuracy, low error, and strong generalization capability, making it effective for time series prediction tasks.

My Comments :-

The Things I have learned from this experiment are as following :-

1. Creation and normalization of time series data.
2. In class I learned that the RNN model works sequentially how a single neuron processes data and gives output
3. While training the model I learned that we focus more on loss rather than accuracy, this model shows how loss is decreasing and that shows that the model is learning data properly
4. The Graphs show how loss decreased till 13 epochs after that it was nearly constant till 30 epochs and similar insights from these few graphs .

